

Week 09

**Write a program for character recognition using RNN and
compare it with CNN**

Aim:

- ❖ To implement and compare the performance of character recognition models using Recurrent Neural Networks (RNN) and Convolutional Neural Networks (CNN) in Python. Aims to analyze how each model type handles character recognition tasks, particularly focusing on the sequential nature of RNNs versus the spatial feature extraction capabilities of CNNs, using the MNIST handwritten digit dataset as a benchmark.

For Understanding

- ❖ **MNIST** stands for Modified National Institute of Standards and Technology dataset.
- ❖ It is one of the most famous datasets in machine learning and deep learning for testing and benchmarking algorithms.
 - ❖ Images: Handwritten digits (0–9).
 - ❖ Image size: Each image is 28×28 pixels (grayscale).
 - ❖ Number of images:
 - ❖ 60,000 training images
 - ❖ 10,000 test images

Step-by-step explanation of the character recognition experiment using RNN and CNN models:

Step 1: Import Libraries

Step 2: Load and Preprocess the MNIST Dataset

Step 3: Reshape Data for RNN Model

Step 4: Build the RNN Model

Step 5: Train the RNN Model

Step 6: Evaluate the RNN Model

Step 7: Reshape Data for CNN Model

Step 8: Build the CNN Model

Step 9: Train the CNN Model

Step 10: Evaluate the CNN Model

Step 11: Compare Results

Import Libraries

1. `import numpy as np`
2. `import matplotlib.pyplot as plt`
3. `import tensorflow as tf`
4. `from tensorflow.keras.datasets import mnist`
5. `from tensorflow.keras.utils import to_categorical`
6. `from tensorflow.keras.models import Sequential`
7. `from tensorflow.keras.layers import Dense, Flatten, Reshape, SimpleRNN, LSTM, GRU, Conv2D, MaxPooling2D`

Load and Preprocess the MNIST Dataset

1. `(x_train, y_train), (x_test, y_test) = mnist.load_data()`
2. `x_train, x_test = x_train / 255.0, x_test / 255.0 # Normalize to [0, 1] range`
3. `y_train, y_test = to_categorical(y_train), to_categorical(y_test) # One-hot encode labels`

Reshape Data for RNN Model

1. `# Reshape data for RNN (sequence of 28x28 images)`
2. `x_train_rnn = x_train.reshape((-1, 28, 28))`
3. `x_test_rnn = x_test.reshape((-1, 28, 28))`

❖ `x_train.reshape((-1, 28, 28))`

❖ -1 → NumPy calculates how many samples there are (60,000 here).

❖ 28, 28 → fixed dimensions for each sample.

❖ So, it becomes (60000, 28, 28).

Build the RNN Model

Model 1: RNN for character recognition (Simple RNN)

1. `rnn_model = Sequential([`
2. `SimpleRNN(128, input_shape=(28, 28), activation='relu'),`
3. `Dense(10, activation='softmax')`
4. `])`
5. `rnn_model.compile(optimizer='adam', loss='categorical_crossentropy',
 metrics=['accuracy'])`

Train the RNN Model

Train RNN model

1. `rnn_model.fit(x_train_rnn, y_train, epochs=5, batch_size=128,
 validation_split=0.1, verbose=1)`

```
In [5]: rnn_model.fit(x_train_rnn, y_train, epochs=5, batch_size=128, validation_split=0.1, verbose=1)
```

Epoch 1/5

422/422 [=====] - 17s 30ms/step - loss: 0.5569 - accuracy: 0.8165 - val_loss: 0.2092 - val_accuracy: 0.9353

Epoch 2/5

422/422 [=====] - 13s 32ms/step - loss: 0.2011 - accuracy: 0.9411 - val_loss: 0.1472 - val_accuracy: 0.9587

Epoch 3/5

422/422 [=====] - 13s 31ms/step - loss: 0.1538 - accuracy: 0.9550 - val_loss: 0.1421 - val_accuracy: 0.9663

Epoch 4/5

422/422 [=====] - 13s 30ms/step - loss: 0.1346 - accuracy: 0.9604 - val_loss: 0.0996 - val_accuracy: 0.9730

Epoch 5/5

422/422 [=====] - 15s 34ms/step - loss: 0.1184 - accuracy: 0.9653 - val_loss: 0.1209 - val_accuracy: 0.9675

```
Out[5]: <keras.src.callbacks.History at 0x1a413b94730>
```

Evaluate the RNN Model

1. `rnn_loss, rnn_accuracy = rnn_model.evaluate(x_test_rnn, y_test, verbose=1)`

```
In [6]: rnn_loss, rnn_accuracy = rnn_model.evaluate(x_test_rnn, y_test, verbose=1)
```

```
313/313 [=====] - 2s 6ms/step - loss: 0.1243 - accuracy: 0.9648
```

Model 2 Reshape Data for CNN Model

1. `x_train_cnn = x_train.reshape((-1, 28, 28, 1))`
2. `x_test_cnn = x_test.reshape((-1, 28, 28, 1))`
3. `cnn_model = Sequential([`
4. `Conv2D(32, kernel_size=(3, 3), activation='relu', input_shape=(28, 28, 1)),`
5. `MaxPooling2D(pool_size=(2, 2)),`
6. `Conv2D(64, kernel_size=(3, 3), activation='relu'),`
7. `MaxPooling2D(pool_size=(2, 2)),`
8. `Flatten(),`
9. `Dense(128, activation='relu'),`
10. `Dense(10, activation='softmax')`
11. `])`
12. `cnn_model.compile(optimizer='adam', loss='categorical_crossentropy',
 metrics=['accuracy'])`

Build the CNN Model

1. `cnn_model.fit(x_train_cnn, y_train, epochs=5, batch_size=128, validation_split=0.1, verbose=1)`

```
In [8]: cnn_model.fit(x_train_cnn, y_train, epochs=5, batch_size=128, validation_split=0.1, verbose=1)
```

Epoch 1/5

422/422 [=====] - 66s 149ms/step - loss: 0.2279 - accuracy: 0.9341 - val_loss: 0.0601 - val_accuracy: 0.9838

Epoch 2/5

422/422 [=====] - 55s 131ms/step - loss: 0.0601 - accuracy: 0.9819 - val_loss: 0.0431 - val_accuracy: 0.9872

Epoch 3/5

422/422 [=====] - 51s 122ms/step - loss: 0.0413 - accuracy: 0.9871 - val_loss: 0.0417 - val_accuracy: 0.9873

Epoch 4/5

422/422 [=====] - 51s 121ms/step - loss: 0.0308 - accuracy: 0.9902 - val_loss: 0.0364 - val_accuracy: 0.9893

Epoch 5/5

422/422 [=====] - 97s 230ms/step - loss: 0.0247 - accuracy: 0.9920 - val_loss: 0.0318 - val_accuracy: 0.9887

```
Out[8]: <keras.src.callbacks.History at 0x1a415fcadf0>
```

Evaluate CNN model

1. `cnn_loss, cnn_accuracy = cnn_model.evaluate(x_test_cnn, y_test, verbose=1)`

```
In [9]: cnn_loss, cnn_accuracy = cnn_model.evaluate(x_test_cnn, y_test, verbose=1)
```

```
313/313 [=====] - 4s 11ms/step - loss: 0.0312 - accuracy: 0.9896
```

Compare the results

1. `print("\nRNN Model Accuracy: {:.2f}%".format(rnn_accuracy * 100))`
2. `print("CNN Model Accuracy: {:.2f}%".format(cnn_accuracy * 100))`

```
In [10]: print("\nRNN Model Accuracy: {:.2f}%".format(rnn_accuracy * 100))
```

```
RNN Model Accuracy: 96.48%
```

```
In [11]: print("CNN Model Accuracy: {:.2f}%".format(cnn_accuracy * 100))
```

```
CNN Model Accuracy: 98.96%
```

Pick one sample from test set

1. `sample_index = 1234` # you can change this number
2. `test_image = x_test[sample_index]`
3. `true_label = np.argmax(y_test[sample_index])`

RNN Prediction

1. `rnn_pred = rnn_model.predict(x_test_rnn[sample_index].reshape(1, 28, 28))`
2. `rnn_pred_label = np.argmax(rnn_pred)`

1/1 [=====] - 0s 43ms/step

CNN Prediction

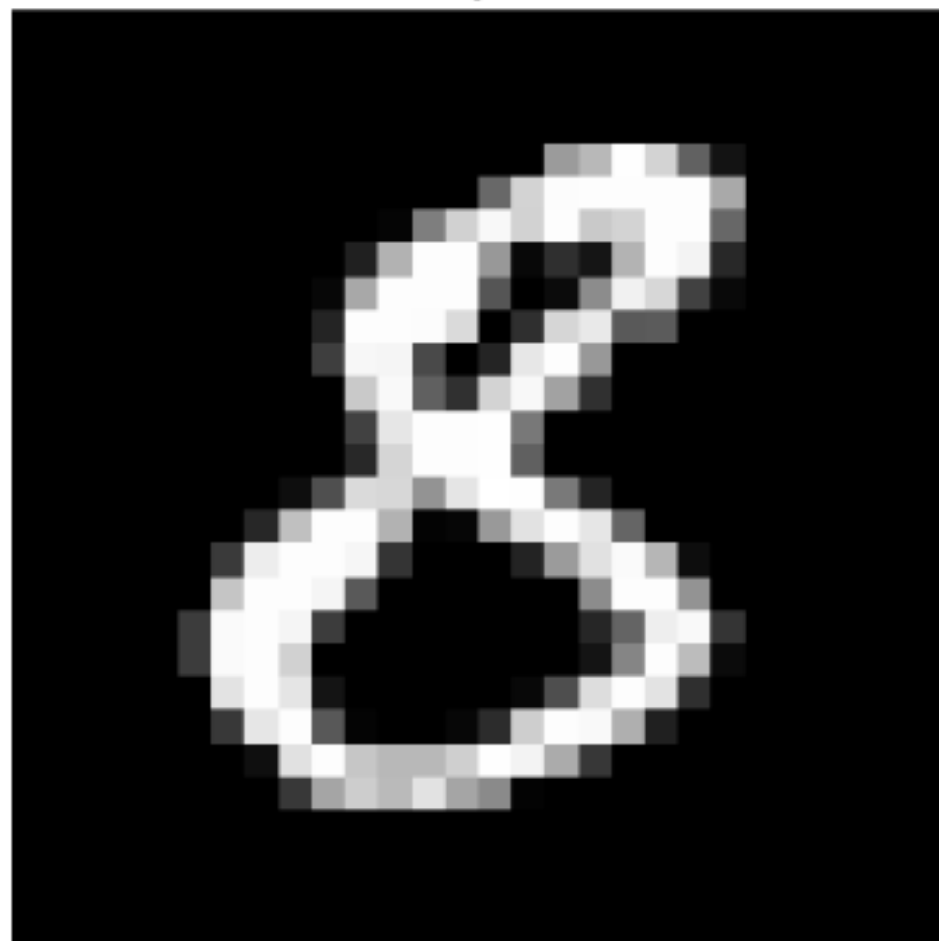
1. `cnn_pred = cnn_model.predict(x_test_cnn[sample_index].reshape(1, 28, 28, 1))`
2. `cnn_pred_label = np.argmax(cnn_pred)`

1/1 [=====] - 0s 71ms/step

Display Results

1. `plt.imshow(test_image, cmap="gray")`
2. `plt.title(f"True Label: {true_label}\nRNN Prediction: {rnn_pred_label} | CNN Prediction: {cnn_pred_label}")`
3. `plt.axis("off")`
4. `plt.show()`

True Label: 8
RNN Prediction: 8 | CNN Prediction: 8



Conclusion

- ❖ In this experiment, we found that CNNs outperform RNNs for character recognition on image-based data like the MNIST dataset. CNNs, designed to capture spatial features, excel at recognizing visual patterns such as edges and shapes, making them ideal for static image tasks. RNNs, while capable of processing sequential data by treating each row of an image as a sequence, are not as effective in extracting spatial hierarchies within images. CNNs achieved higher accuracy in fewer epochs, indicating their efficiency and suitability for character recognition tasks. Overall, CNNs are preferred for image-based character recognition, whereas RNNs are better suited for sequential tasks, such as handwriting or speech recognition.